Day 1 - Variables, Types, and the Object Model

Day 1 — Variables, Types, and the Object Model

If you haven't read `day00-primer/lesson.md` yet, read that first — it covers what a terminal, a script, and the REPL are, which everything below assumes you already know.

Objectives

- Understand what a Python variable actually *is* — a name pointing at a value, not a labeled box holding it
- Meet the core built-in data types and understand what each one is *for*
- Understand the difference between two things that look similar but aren't: values that can change after creation ("mutable") and values that can't ("immutable"), and why this distinction causes real, common bugs
- Understand why Python lets a variable hold a number one moment and text the next ("dynamic typing")

What is a variable, really?

In everyday language, we might say "let `x` be 5" — and in most programming tutorials, that's illustrated as a labeled box: a box named `x`, with the number `5` inside it. That picture is close enough to get started with simple numbers, but it will actively mislead you once you meet lists and dictionaries later today — so let's replace it with the correct picture from the start.

Here's what actually happens when Python runs this line:

```
x = 5
```

1. Python creates an **object** in the computer's memory representing the value `5`. (An "object" is just Python's word for "a piece of data floating around in memory" — every single value in Python, from numbers to text to entire programs, is an object. You'll see why this matters more and more as the course goes on.)
2. Python creates a **name**, `x`, and points it at that object — like a sticky note with "x" written on it, stuck onto the `5` object.

That's it. A variable is a **name that points at an object** — not a container that holds a copy of the value. This distinction feels academic right now, with plain numbers, but watch what happens when two names point at the *same* object:

```
a = [1, 2, 3]   # a list -- more on lists later today
b = a           # b now points at the SAME object a points at -- not a copy!
a.append(4)     # this changes the object that a (and b) point to
print(b)        # [1, 2, 3, 4]  <- surprising if you were picturing "boxes"!
```

If you were thinking of `a` and `b` as separate boxes, this result looks like a bug — `b` was never supposed to change, you never touched `b`! But once you picture `a` and `b` as two sticky notes stuck on the *same* physical list, the result makes total sense: there was only ever one list in memory. `a.append(4)` changed that one list, and both sticky notes ("names") see the same, single, changed object, because they were always pointing at the same thing.

This is going to come up constantly for the rest of this course, so let's build a tool to check it for ourselves.

Proving it with `id()`

Python has a built-in function called `id()` (a function is a named, reusable operation you can invoke by writing its name followed by parentheses — you'll learn to write your own on Day 4; for now you're just using ones Python already provides). `id(some_value)` returns a unique number identifying exactly which object in memory that value refers to — think of it as that object's "physical address" or serial number.

```
a = [1, 2, 3]
b = a
print(id(a))       # some number, e.g. 140185732791872
print(id(b))       # the EXACT SAME number -- proving a and b point at the same object
print(id(a) == id(b))   # True
```

Python gives you a shortcut for exactly this comparison: the `is` operator, which asks "are these two names pointing at the literal same object?" (as opposed to "do these two objects merely look equal?", which is what `==` asks):

```
print(a is b)   # True -- same object
c = [1, 2, 3]    # a brand NEW list that just happens to contain the same values
print(a is c)    # False -- two different objects in memory
print(a == c)    # True -- but their CONTENTS are equal
```

Keep `is` (identity: "the literal same object?") and `==` (equality: "same value/contents?") mentally separate — mixing them up is a classic beginner bug, especially once you learn about `None` on Day 2, where you'll be told to always write `x is None` rather than `x == None` (both usually work, but `is` is the semantically correct and universally-used idiom for this specific check).

Why does Python work this way? (the practical reason, not just trivia)

Imagine every single time you wrote `b = a`, Python secretly made a full, independent copy of whatever `a` pointed to. For a single number that's cheap and harmless. But what if `a` were a list with ten million items in it? Copying it on every single assignment, function call, and pass into another variable would make your programs slow and would use huge amounts of memory for no reason, most of the time. So Python's design instead just copies the *pointer* (the name-to-object link) — which is always cheap, no matter how big the underlying object is — and only makes an actual copy when *you* explicitly ask for one. You'll learn exactly how to ask for a copy later today (`.copy()`, `list(...)`, or slicing).

Mutable vs. immutable — the single most important distinction in this entire language

This is the concept that, if you truly understand it today, will save you hours of confusion in the coming weeks. Every type of value in Python falls into exactly one of these two categories:

- **Immutable** — once created, this object can *never* be changed. Any operation that looks like it's "modifying" it is secretly creating a brand new object instead. Types: `int` (whole numbers), `float` (decimal numbers), `bool` (True/False), `str` (text), `tuple` (a fixed, ordered group of values — introduced properly on Day 3), `frozenset`.

- **Mutable** — this object *can* be changed in place, after it's created, without creating a new object. Types: `list`, `dict`, `set` (all introduced Day 3), and any custom class you write yourself (Day 8) unless you deliberately design it not to be.

Watch this carefully:

```
x = 5
x = 6
```

It's tempting to describe this as "we changed `x` from 5 to 6." But that's not quite what happened, and the precise version matters: **integers are immutable — the object `5` was never changed (it can't be — nothing can change it).** What actually happened: Python created a brand new object, `6`, and moved the name `x` to point at that new object instead. The old `5` object still exists somewhere in memory for a moment, completely untouched, until Python notices nothing points at it anymore and cleans it up automatically (this automatic cleanup process is called "garbage collection" — you don't need to manage it yourself, just know that it happens).

Now compare with a mutable type:

```
s = "hello"
s[0] = "H"     # this raises: TypeError: 'str' object does not support item assignment
```

Strings are immutable — you cannot change a single character of an existing string object; Python won't let you, and raises an error the moment you try. If you want an uppercase version, you must create an *entirely new* string:

```
s = "hello"
s = "Hello"    # this doesn't modify the old string -- it creates a NEW string object
               # and points s at it instead. The old "hello" object is discarded.
```

Now let's see the mutable case, where in-place change genuinely is possible:

```
lst = [1, 2, 3]
lst.append(4)   # this DOES modify the existing list object, in place
print(lst)       # [1, 2, 3, 4] -- same object, now with different contents
```

`lst.append(4)` really does reach into the existing list object sitting in memory and add an item to it — no new object is created. This is the behavior that made `b` change when we only touched `a`, in the very first example of this lesson.

Why this matters when you pass values into functions

You haven't formally learned functions yet (that's Day 4), but you've likely already used `print(...)`, and you'll be writing your own simple ones very soon, so it's worth planting this seed now: when you hand a value into a function, Python copies the *name-to-object pointer* into the function, exactly like `b = a` above — never the underlying object itself. This means:

- If the function does something that mutates the object in place (like `.append(...)` on a list), the caller's variable sees that change too, because there was only ever one object.
- If the function just does `parameter_name = something_else` (pointing the *local* name at a different object), that has **zero** effect on the caller's variable — the caller's name still points at whatever it originally pointed at.

You'll see this demonstrated concretely, with real code you can run and predict yourself, in today's exercises (`mutate_vs_reassign_demo`) — and you'll return to this exact idea in much more depth on Day 4 once you're writing functions of your own.

The core built-in types — your basic vocabulary of data

Every piece of data in Python has a **type** — the type tells Python (and you) what *kind* of thing a value is, and therefore what you're allowed to do with it. You find out any value's type using the built-in `type()` function:

```
type(5)          # <class 'int'>
type(5.0)         # <class 'float'>
type("hi")         # <class 'str'>
type([1, 2])        # <class 'list'>
```

Type	Example	Mutable?	What it's for
`int`	`42`	No	Whole numbers, positive or negative, no decimal point. Python's integers have no size limit (unlike many languages) — you can have arbitrarily huge whole numbers without special handling.
`float`	`3.14`	No	Numbers with a decimal point. Internally stored in a format ("IEEE 754 double") that can't represent every decimal exactly, which is why `0.1 + 0.2` prints `0.30000000000000004` instead of exactly `0.3` — this is a well-known quirk of nearly every programming language, not a Python bug.

`bool`	`True`	No	Represents yes/no, true/false. Under the hood, `bool` is actually a special case of `int` — `True` behaves exactly like `1` and `False` like `0` in arithmetic (`True + True == 2` is actually true — try it in the REPL).
`str`	`"hi"`	No	Text ("string" is short for "string of characters"). Written between either single `'...'` or double `"..."` quotes — Python treats them identically, just be consistent within one file.
`list`	`[1, 2]`	**Yes**	An ordered, changeable collection of values, in square brackets. Full treatment Day 3.
`tuple`	`(1, 2)`	No	An ordered, UNchangeable collection — like a list that's locked once created. Full treatment Day 3.
`dict`	`{"a": 1}`	**Yes**	Short for "dictionary" — maps keys to values, like a real dictionary maps words to definitions. Full treatment Day 3.
`set`	`{1, 2}`	**Yes**	An unordered collection with no duplicate values allowed. Full treatment Day 3.

`NoneType`	`None`	—	`None` is Python's way of representing "no value" or "nothing here" — similar to what other languages call "null." There is exactly one `None` object in the whole running program; every variable set to `None` points at that same single object.

You don't need to memorize this table — you'll use every one of these types so often over the next two weeks that they'll become second nature. Treat it as a reference to glance back at.

To check "is this value of type X," prefer `isinstance(value, X)` over comparing `type(value) == X` directly:

```
isinstance(5, int)      # True
isinstance(True, int)    # True -- bool is considered a "kind of" int, as explained above
```

`isinstance` is the idiomatic (meaning: the way experienced Python programmers actually write it) choice because, as you'll see on Day 9, it correctly handles categories of related types, not just one exact type — `type(x) == X` gets this subtly wrong in ways that don't matter yet, but will once you're writing your own classes.

Dynamic typing: a variable's type can change, because a variable is just a name

```
x = 5          # right now, the name x points at an int object
x = "hello"    # now x points at a completely different object -- a str
```

This is completely legal in Python and is called **dynamic typing**: a variable isn't locked to one type forever — it can point at any type of object, and which type it currently points to is only decided by whatever it's currently pointing at, checked while the program is actually running (as opposed to some languages, where you must declare "x is an integer, forever" up front, and the tool that turns your code into a runnable program will refuse to run it if you ever assign text to it later).

This flexibility is convenient, but it has a real cost: Python can't warn you in advance that you're about to misuse a variable (say, treating a string like a number) — that mistake will only surface when the misbehaving line actually runs, possibly deep inside a program that otherwise looked fine. This is a large part of *why* Day 12 (testing) matters so much professionally — tests are how Python programmers compensate for the lack of an upfront check that other languages get for free.

Truthiness — what counts as "true enough" in an `if`

You haven't learned `if` yet in full (that's Day 2, tomorrow) but since it's so central, here's a preview relevant to today's types: every single value in Python, of any type, has an implicit yes/no answer to "is this true-ish?"

when used somewhere expecting True/False, such as inside an `if`. The values that count as "false-ish" (called **falsy**) are, essentially, all the ways of representing "empty" or "nothing":

`False`, `None`, `0`, `0.0`, `""` (empty string), `[]` (empty list), `{}` (empty dict), and a few others you'll meet later (empty tuple, empty set). **Every other value is truthy** — including things that might surprise you at first, like the string `"False"` (a non-empty string containing the *word* "False" is still just a non-empty string, and is therefore truthy) or the number `-1` (nonzero, so truthy).

```
if []:
    print("this line never runs -- an empty list is falsy")
if [0]:
    print("this line DOES run -- a list containing one item (even the falsy number 0) is itself non
```

Exercises

Open `exercises.py` in this same folder and implement each function where you see `# TODO`. Run the file from your terminal with `python exercises.py` and read the PASS/FAIL output. If anything says FAIL, that function isn't doing what its docstring describes yet — fix it and run again. Once everything passes, open `solutions.py` to compare your approach.